TI2 - Sprint 4

Smart Routine

Gestão Inteligente do Cotidiano

- Alex de Castro Mendes Marques
- Gabriel Ribeiro Souza Silva
- Karen Cristina Satiro da Costa

Problema Identificado

- Pessoas esquecem alimentos na despensa
- Muitos produtos vencem ou são desperdiçados
- Falta de controle do que foi comprado
- Dificuldade em decidir o que cozinhar com o que se tem
- Falta de sistemas simples que ajudem na rotina alimentar

- Conclusão:
  → Desperdício de comida
  → Perda financeira
  → Falta de organização

Nossa Solução: Smart Routine

- ✓ Controlar a despensa
- ✓ Gerar receitas usando IA (Qwen 2.5 – Hugging Face)
- ✓ Criar e compartilhar receitas
- ✓ Planejamento alimentar automático
- ✓ Reduz desperdício

Funcionalidade Chave	Benefício Direto
Controle de Inventário e Alertas de Validade Proativos	**Econômico:** Evita compras duplicadas e descarte de alimentos.
Sugestão Inteligente de Receitas	**Prático:** Prioriza ingredientes próximos ao vencimento, facilitando o planejamento.
Hábitos Sustentáveis	**Ambiental:** Reduz a pegada de carbono e promove o Consumo Responsável.

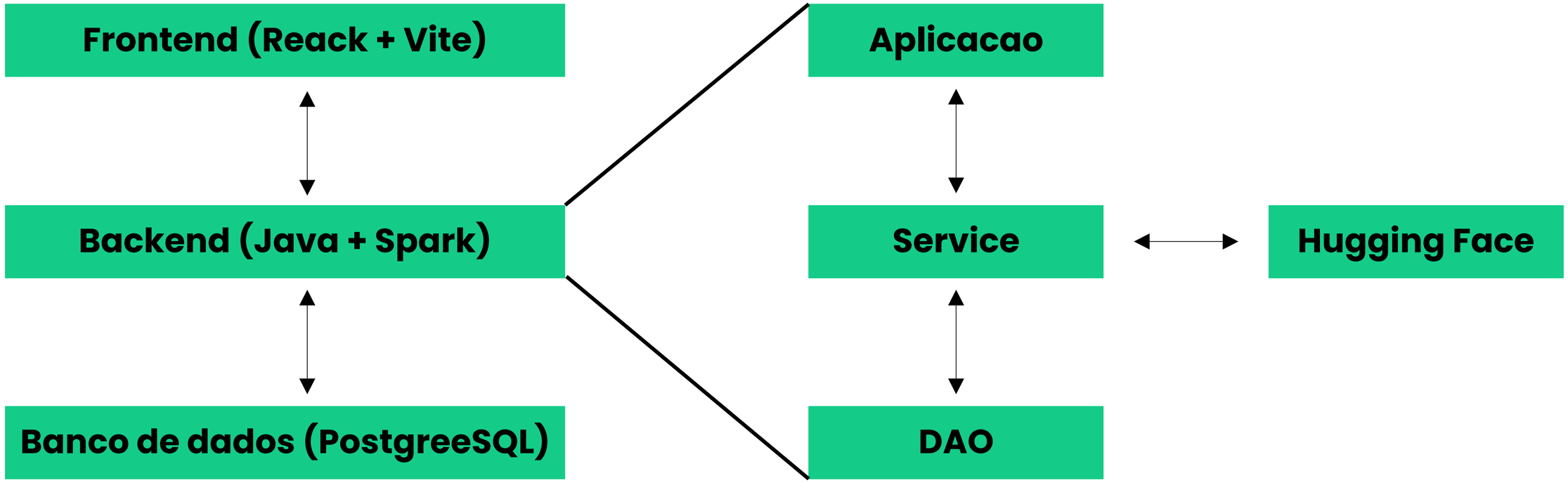
Requisitos Funcionais

- ✅ Cadastro/login
- ✅ Gerenciar despensa
- ✅ Listar alimentos do usuário
- ✅ Gerar receita com IA usando seus alimentos
- ✅ Favoritar receitas
- ✅ Criar receita manual
- ✅ Compartilhar receitas
- ✅ Notificação de validade de itens
- ✅ Edição de alimentos

Requisitos não funcionais

- Segurança (BCrypt, variáveis ambiente)
- Usabilidade
- Desempenho
- Disponibilidade
- Compatibilidade
- Escalabilidade

Arquitetura do sistema



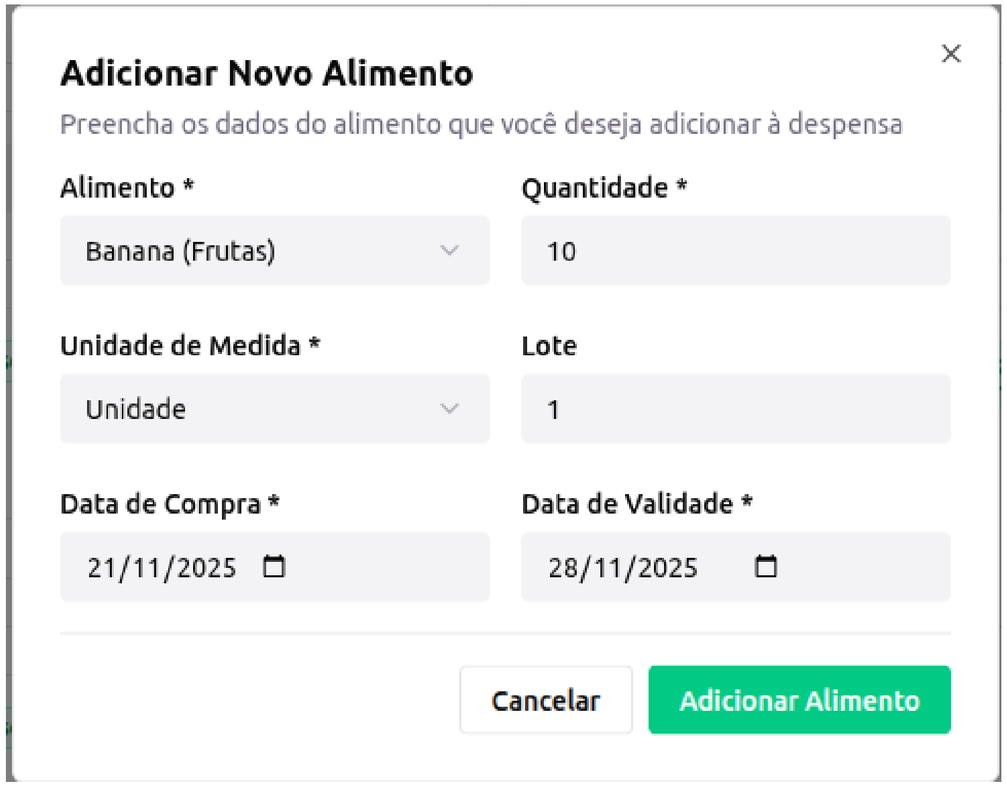
Proteção de Dados

```typescript
export async function hashPasswordWebCrypto(password: string): Promise<string> {
  const encoder = new TextEncoder();
  const data = encoder.encode(password);
  const hashBuffer = await crypto.subtle.digest('SHA-256', data);
  const hashArray = Array.from(new Uint8Array(hashBuffer));
  const hashHex = hashArray.map(b => b.toString(16).padStart(2, '0')).join('');
  return hashHex;
}
```

```java
public boolean insert(Usuario usuario) {  Karen Satiro +1
    boolean status = false;

    // Gera o HASH da senha antes de salvar
    String senhaHashed = BCrypt.hashpw(usuario.getSenha(), BCrypt.gensalt());

    // SQL Injection protegido com '?'
    String sql = "INSERT INTO usuario (nome, email, senha, data_nascimento, data_adicao) VALUES (?, ?, ?, ?, ?)";

    try {
        PreparedStatement ps = conexao.prepareStatement(sql);
        ps.setString( parameterIndex: 1, usuario.getNome());
        ps.setString( parameterIndex: 2, usuario.getEmail());
        ps.setString( parameterIndex: 3, senhaHashed);
        ps.setDate( parameterIndex: 4, Date.valueOf(usuario.getDataNascimento()));
        ps.setDate( parameterIndex: 5, Date.valueOf(usuario.getDataAdicao()));

        ps.executeUpdate();
        ps.close();
```

Smart Routine

Adicionando alimento – Front




```
<DialogContent className="max-w-2xl max-h-[90vh] overflow-y-auto">
    <DialogHeader>
        <DialogTitle>Adicionar Novo Alimento</DialogTitle>
        <DialogDescription>
            Preencha os dados do alimento que você deseja adicionar à despensa
        </DialogDescription>
    </DialogHeader>
    <form onSubmit={handleAddFood} className="space-y-4">
        <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
            <div className="space-y-2">
                <Label htmlFor="alimento">Alimento *</Label>
                <Select
                    value={formData.alimentoId}
                    onValueChange={(value) => handleFormChange('alimentoId', value)}
                    required
                >
                    <SelectTrigger>
                        <SelectValue placeholder="Selecione o alimento" />
                    </SelectTrigger>
                    <SelectContent>
                        {alimentos.map((alimento) => (
                            <SelectItem key={alimento.id} value={alimento.id.toString()}>
                                {alimento.nome} ({alimento.categoria})
                            </SelectItem>
                        ))}
                    </SelectContent>
                </Select>
            </div>
```

Smart Routine

Adicionando alimento – Back

AlimentoService.java

Aplicacao.java

```java
path( path: "/alimento", () -> {
    get( path: "/search", alimentoService::search);
    get( path: "/categorias", alimentoService::getCategorias);
    get( path: "/categoria/:categoria", alimentoService::getByCategoria);
    get( path: "/:id", alimentoService::get);
    get( path: "", alimentoService::getAll);
    post( path: "", alimentoService::insert);
    put( path: "/:id", alimentoService::update);
    delete( path: "/:id", alimentoService::delete);
});
```

```java
public Object insert(Request request, Response response) {  Karen Satiro
    response.type( contentType: "application/json");

    try {
        Alimento alimento = gson.fromJson(request.body(), Alimento.class);

        // Validações
        if (alimento.getNome() == null || alimento.getNome().trim().isEmpty()) {
            response.status( statusCode: 400);
            return gson.toJson(new ErrorResponse("Nome é obrigatório"));
        }

        if (alimentoDAO.insert(alimento)) {
            response.status( statusCode: 201);
            return gson.toJson(new SuccessResponse("Alimento cadastrado com sucesso"));
        } else {
            response.status( statusCode: 500);
            return gson.toJson(new ErrorResponse("Erro ao cadastrar alimento"));
        }
    } catch (Exception e) {
        response.status( statusCode: 400);
        return gson.toJson(new ErrorResponse("Dados inválidos: " + e.getMessage()));
    }
}
```

Smart Routine

Adicionando alimento – Back

AlimentoDAO.java

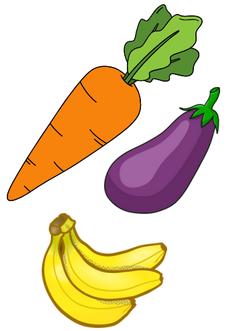
```java
public boolean insert(Alimento alimento) {
    boolean status = false;
    String sql = "INSERT INTO alimento (nome, categoria) VALUES (?, ?)";

    try {
        PreparedStatement ps = conexao.prepareStatement(sql);
        ps.setString( parameterIndex: 1, alimento.getNome());
        ps.setString( parameterIndex: 2, alimento.getCategoria());

        ps.executeUpdate();
        ps.close();
        status = true;
    } catch (SQLException e) {
        System.err.println("Erro ao inserir alimento: " + e.getMessage());
    }
    return status;
}
```

Banco de dados

id	nome	categoria
1	Leite Integral	Laticínios
2	Leite Desnatado	Laticínios
3	Queijo Mussarela	Laticínios
4	Iogurte Natural	Laticínios
5	Manteiga	Laticínios
6	Requeijão	Laticínios
7	Arroz Branco	Grãos
8	Arroz Integral	Grãos
9	Feijão Preto	Grãos
10	Feijão Carioca	Grãos
11	Lentilha	Grãos
12	Macarrão Espaguete	Massas
13	Macarrão Penne	Massas
14	Tomate	Vegetais
15	Cebola	Vegetais
16	Alho	Temperos
17	Cenoura	Vegetais
18	Batata	Vegetais
19	Brócolis	Vegetais
20	Alface	Vegetais
21	Pimentão Vermelho	Vegetais
22	Banana	Frutas

Smart Routine

Implementação da IA

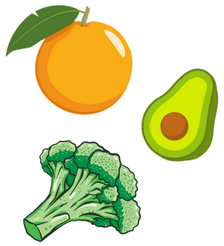


Serviço Utilizado

O sistema utiliza o modelo Qwen/Qwen2.5-7B-Instruct através do Hugging Face para geração dinâmica de receitas.

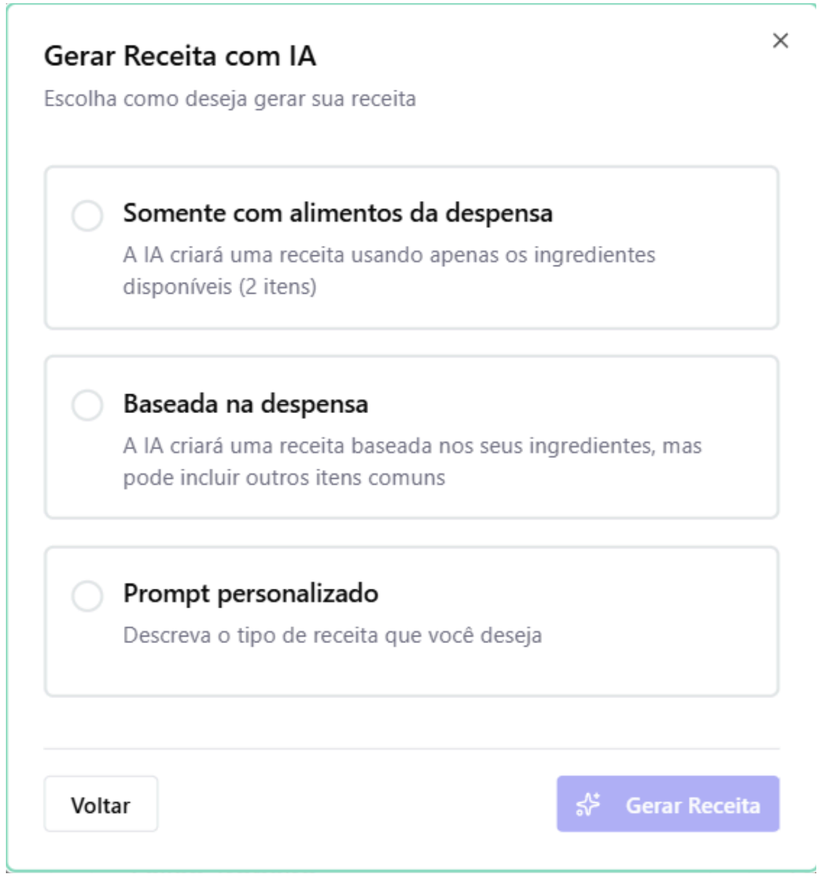
Funcionamento

- O Backend recupera os alimentos do usuário do Banco de Dados.
- O sistema monta um prompt: "Crie uma receita usando apenas: [Lista de Alimentos]".
- A API retorna a receita estruturada.
- A nova receita é salva no banco de dados



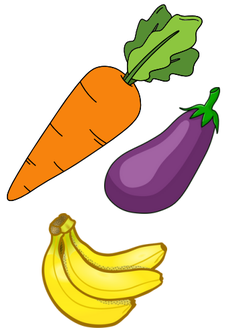
Smart Routine

Utilização da IA




```java
private JsonObject callHF(JsonObject payload) throws IOException {  1 usage  Gabriel Ribeiro
    System.out.println("AI PAYLOAD");
    System.out.println(payload);

    URL url = new URL(HF_URL);
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("POST");
    conn.setRequestProperty("Authorization", "Bearer " + HF_TOKEN);
    conn.setRequestProperty("Content-Type", "application/json");
    conn.setDoOutput(true);
    byte[] out = payload.toString().getBytes(StandardCharsets.UTF_8);
    try (OutputStream os = conn.getOutputStream()) { os.write(out); }
    int code = conn.getResponseCode();
    InputStream is = code >= 200 && code < 300 ? conn.getInputStream() : conn.getErrorStream();
    String resp = new String(is.readAllBytes(), StandardCharsets.UTF_8);
    return JsonParser.parseString(resp).getAsJsonObject();
}
```

Intelligent System Canvas



Ferramental de IA

- Qwen 2.5 – Hugging Face
- Azure OpenAI Service;
- Azure App Service.

Entradas

Prompt de texto estruturado contendo a lista de ingredientes do inventário do usuário e o catálogo de receitas do nosso banco de dados.

Saídas

Resposta em formato JSON com as melhores sugestões de receitas.

Proposição de Valor

- Prevenção de Desperdício;
- Organização e Planejamento;
- Inteligência Aplicada (Futura);
- Promove uma Alimentação Mais Saudável e Criativa.

Equipe

- Desenvolverdores(as) Full-Stack;

Stakeholders Chaves

- Investidores e parceiros financeiros;
- Parceiros de ecossistema (empresas e influenciadores, nutricionistas);
- Usuário final (pessoas e famílias).

Clientes

- Famílias e indivíduos focados em otimizar o orçamento doméstico;
- Usuários interessados em alimentação saudável e sustentável;
- Profissionais e pais com rotinas corridas que valorizam a conveniência.

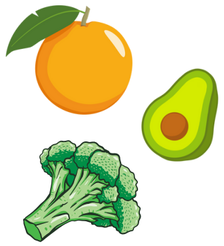
Smart Routine

Custos

- Custos de Infraestrutura (Azure) e servidores;
- Marketing e divulgação;
- Suporte e manutenção;
- Custos associados a parcerias.

Receitas

- Modelo Freemium: versão gratuita básica e versão paga com recursos avançados (relatórios, acesso ilimitado as recomendações personalizadas);
- Publicidade segmentada com foco em sustentabilidade e parcerias;
- Parcerias de Afiliados;
- Marketplace de Receitas (Visão de Futuro).



OBRIGADO!

Estamos abertos a perguntas e demonstrações ao vivo

Smart Routine